



Universidad Internacional del Ecuador

Carrera

Ingeniería en Ciberseguridad

Materia

Lógica de programación

Tema

Actividad de contacto con el docente

Alumno

Frans Alexander Catagña Lugmaña

Fecha

28 de junio de 2026



ÍNDICE

1. Fase 1: Entendimiento

Generador seguro de contraseñas

a. ¿Cuál es el problema?

El proyecto consiste en desarrollar una aplicación que permita generar contraseñas seguras y aleatorias para proteger cuentas de usuarios, sistemas informáticos y plataformas digitales, el sistema ayudará a los usuarios a crear contraseñas seguras mediante la combinación automática de letras mayúsculas, letras minúsculas, números y caracteres especiales.

b. ¿Quién tiene el problema?

Estudiantes, docentes, empresas, usuarios de internet y organizaciones.

c. ¿Por qué es importante resolver el problema?

Porque una contraseña segura es la primera barrera de protección para la información digital.

d. Propuesta de solución

Desarrollar un software que genere contraseñas aleatorias y segura utilizando letras, números y caracteres especiales.

2. Fase 2: Planeación

Objetivo General

Desarrollar un sistema capaz de generar contraseñas seguras y aleatorias para mejorar la protección de cuentas y sistemas digitales.

Objetivos Específicos

- Solicitar la longitud de la contraseña.
- Validar los datos ingresados.
- Generar una contraseña aleatoria.
- Mostrar el resultado.
- Permitir generar nuevas contraseñas

a. Cuadro de análisis del problema

Entradas	Procesamiento	Salidas
Longitud de la contraseña	Validar longitud	Contraseña generada
Selección de caracteres	Crear conjunto de caracteres	Mensajes de validación
Opción continuar o salir	Generar caracteres aleatorios	Resultado final
Preferencias del usuario	Construir contraseña	Confirmación de operación



3. Fase 3: Diseño



```
1  import secrets
2  import string
3
4
5  # =====
6  # VARIABLES GLOBALES
7  # =====
8
9  # Lista donde se almacenarán las contraseñas generadas
10 historial = []
11
12 # Tupla con los niveles de seguridad según la longitud
13 niveles_seguridad = (
14     "Baja",
15     "Media",
16     "Alta"
17 )
18
19 # Diccionario que contiene los diferentes grupos de caracteres
20 # que pueden utilizarse para generar la contraseña
21 caracteres = {
22     "minusculas": string.ascii_lowercase,
23     "mayusculas": string.ascii_uppercase,
24     "numeros": string.digits,
25     "simbolos": string.punctuation
26 }
27
28
29 # =====
30 # FUNCIONES
31 # =====
32
```



```
33 def evaluar_seguridad(longitud):
34     """
35     Determina el nivel de seguridad de una contraseña
36     en función de su longitud.
37     """
38
39     if longitud < 8:
40         return niveles_seguridad[0]
41
42     elif longitud < 12:
43         return niveles_seguridad[1]
44
45     else:
46         return niveles_seguridad[2]
47
48
49 def generar_contrasena(longitud,
50                         usar_mayusculas,
51                         usar_numeros,
52                         usar_simbolos):
53     """
54     Genera una contraseña aleatoria utilizando los
55     tipos de caracteres seleccionados por el usuario.
56     """
57
58     # Siempre se incluyen letras minúsculas
59     conjunto = caracteres["minusculas"]
60
61     # Agrega letras mayúsculas si el usuario lo solicita
62     if usar_mayusculas:
63         conjunto += caracteres["mayusculas"]
```



```
64
65     # Agrega números si el usuario lo solicita
66     if usar_numeros:
67         conjunto += caracteres["numeros"]
68
69     # Agrega símbolos si el usuario lo solicita
70     if usar_símbolos:
71         conjunto += caracteres["símbolos"]
72
73     contraseña = ""
74
75     # Genera la contraseña carácter por carácter
76     # usando la librería secrets para mayor seguridad
77     for _ in range(longitud):
78         contraseña += secrets.choice(conjunto)
79
80     return contraseña
81
82
83 def mostrar_historial():
84     """
85     Muestra todas las contraseñas generadas durante
86     la ejecución del programa.
87     """
88
89     print("\n===== HISTORIAL =====")
90
91     # Verifica si existen contraseñas almacenadas
92     if len(historial) == 0:
93         print("No existen contraseñas generadas.")
```



```
94
95     else:
96         # Recorre e imprime cada contraseña guardada
97         for i, password in enumerate(historial, start=1):
98             print(f"{i}. {password}")
99
100
101 # =====
102 # PROGRAMA PRINCIPAL
103 # =====
104
105 # Bucle principal del programa
106 while True:
107
108     print("\n=====")
109     print(" GENERADOR SEGURO DE CONTRASEÑAS ")
110     print("=====")
111
112     try:
113
114         # Solicita la longitud de la contraseña
115         longitud = int(
116             input("Ingrese la longitud de la contraseña: ")
117         )
118
119         # Valida que la longitud mínima sea 8 caracteres
120         while longitud < 8:
121             print(
122                 "Error: La longitud mínima es 8 caracteres."
123             )
```



```
124         longitud = int(  
125             input("Ingrese nuevamente la longitud: ")  
126         )  
127  
128     # Solicita las características deseadas  
129     mayusculas = input(  
130         "¿Incluir mayúsculas? (s/n): "  
131     ).lower() == "s"  
132  
133     numeros = input(  
134         "¿Incluir números? (s/n): "  
135     ).lower() == "s"  
136  
137     simbolos = input(  
138         "¿Incluir símbolos? (s/n): "  
139     ).lower() == "s"  
140  
141     # Genera la contraseña con las opciones elegidas  
142     password = generar_contrasena(  
143         longitud,  
144         mayusculas,  
145         numeros,  
146         simbolos  
147     )  
148  
149     # Guarda la contraseña en el historial  
150     historial.append(password)  
151  
152     # Muestra la contraseña generada  
153     print("\nContraseña generada:")  
154     print(password)  
155
```




```
156     # Muestra el nivel de seguridad estimado
157     print(
158         f"Nivel de seguridad: "
159         f"{evaluar_seguridad(longitud)}"
160     )
161
162     # Menú de opciones para el usuario
163     opcion = input(
164         "\n1. Generar otra contraseña\n"
165         "2. Ver historial\n"
166         "3. Salir\n"
167         "Seleccione una opción: "
168     )
169
170     # Muestra el historial de contraseñas
171     if opcion == "2":
172         mostrar_historial()
173
174     # Finaliza el programa
175     elif opcion == "3":
176         print("\nPrograma finalizado.")
177         break
178
179     # Captura errores cuando el usuario no ingresa números válidos
180     except ValueError:
181         print(
182             "Error: Debe ingresar un número válido."
183         )
```

4. Fase 4: Revisión del Diseño

Antes de programar verificamos:

Uso de listas

Uso de tuplas

Uso de diccionarios

Uso de funciones

Uso de ciclos

Uso de condicionales

Validación de datos

Manejo de excepciones

Generación aleatoria segura

Historial de contraseñas



Fortalezas del Proyecto

Interfaz sencilla y fácil de utilizar.

Validación de longitud mínima para mejorar la seguridad.

Uso de la librería secrets, recomendada para aplicaciones de seguridad.

Almacenamiento temporal de contraseñas generadas.

Clasificación del nivel de seguridad según la longitud.

Manejo adecuado de errores de entrada.

Limitaciones Identificadas

El historial se pierde al cerrar el programa.

La evaluación de seguridad se basa únicamente en la longitud.

No verifica que la contraseña contenga obligatoriamente todos los tipos de caracteres seleccionados.

No exporta las contraseñas a un archivo externo.

Mejoras Futuras

Guardar el historial en un archivo de texto o base de datos.

Copiar automáticamente la contraseña al portapapeles.

Agregar una barra de fortaleza de contraseña.

Verificar complejidad mediante criterios más avanzados.

Permitir generar múltiples contraseñas simultáneamente.

Crear una interfaz gráfica utilizando Python y Tkinter.

Exportar el historial a formatos como TXT o CSV.

5. Conclusiones

El proyecto Generador Seguro de Contraseñas fue desarrollado y probado satisfactoriamente. Durante la fase de revisión se comprobó que todas las funcionalidades principales operan correctamente, incluyendo la generación aleatoria de contraseñas, la validación de datos, el almacenamiento en historial y la clasificación de seguridad.

Además, el proyecto permitió aplicar conocimientos fundamentales de programación en Python, como estructuras de datos (listas, tuplas y diccionarios), funciones, ciclos, estructuras condicionales, manejo de excepciones y uso de librerías especializadas. Los resultados obtenidos demuestran que la solución cumple con los objetivos planteados y constituye una herramienta funcional para la generación de contraseñas seguras.



Universidad Internacional del Ecuador